assignment4_part3

October 24, 2026

1 Assignment 4: Dealing with Sequences Real-World Data (Part III)

1.1 Detecting Near Duplicates

Sequence similarity metrics can be widely applied to many tasks. In this part of the assignment, let's look at another application of sequence similarity – detecting near-duplicates. Near-duplicate detection is commonly used in plagiarism detection, index selection for search engines, copy-paste detection, biological sequence alignments, and beyond. Edit distance is still a viable solution to these problems, but when we are dealing with longer sequences, the efficiency becomes a big issue. Unfortunately, edit distance has to fill in the n by n table - in other words, it has a time complexity of O(n^2). This can't scale up. Shingling is a much more efficient solution.

Following the introduction of *shingling* in the lecture, let's practice it on our Twitter dataset. The *shingling* approach relies on n-grams and Jaccard similarity. Luckily, we have already been familiar with both.

First, let's import the necessary packages.

[3]:
```
import pandas as pd
import csv
import nltk
from collections import Counter
```

1.1.1 Exercise 4. Shingling (20 pts)

Complete the `shingling_jaccard_similarity` function to compute the similarity score between two pieces of text using the shingling approach. Specifically, you should (1) represent both text sequences as sets of overlapping n-grams (n specified as an argument) and (2) compute the Jaccard similarity between the two sets. We have implemented a `jaccard_similarity` function for your convenience.

Hint: 1. You may use the `nltk.ngrams` API to obtain the n-grams. 2. The `nltk.ngrams` API returns an iterator of tuples. You may wrap it up with `list()` to collect the n-grams as a list. You may checkout how we use `nltk.bigrams` in Part 1 of this assignment as an example.

[4]:
```
tokenizer = nltk.tokenize.casual.TweetTokenizer()

def jaccard_similarity(list_x, list_y):
    set_x = set(list_x)
```

```
    set_y = set(list_y)
    intersection = set_x.intersection(set_y)
    union = set_x.union(set_y)
    return len(intersection) / len(union) if len(union) > 0 else 0

def shingling_jaccard_similarity(text_x, text_y, n):
    # YOUR CODE HERE
    #raise NotImplementedError()
    nigrams_x = list(nltk.ngrams(tokenizer.tokenize(text_x), n))
    nigrams_y = list(nltk.ngrams(tokenizer.tokenize(text_y), n))
    sim_score = jaccard_similarity(nigrams_x, nigrams_y)
    return sim_score
```

You may try out the examples used in the lecture:

[5]:
```
x = "to be or not to be"
y = "not be or not to be"
z = "be or not to not be"

print(shingling_jaccard_similarity(x,y, 3))
print(shingling_jaccard_similarity(x,z, 3))
```

```
0.6
0.3333333333333333
```

[6]:
```
# This code block tests if the `shingling_jaccard_similarity` function is␣
 ↪implemented correctly
# We hide some tests. Passing the displayed assertions does not guarantee full␣
 ↪points.

x = "to be or not to be"
y = "not be or not to be"
z = "be or not to not be"

assert abs(shingling_jaccard_similarity("to be or not to be", "not be or not to␣
 ↪be", 3) - 0.6) < 1e-8
assert abs(shingling_jaccard_similarity("to be or not to be", "be or not to not␣
 ↪be", 3) - 1/3) < 1e-8
```

Now let's see how this similarity function can help us detect near-duplicate Tweets in our data set.

[7]:
```
tweet_df = pd.read_csv("assets/tweets.txt", sep="\t", header=None)
tweet_df.columns = ['text']
```

One of the Tweet that caught our attention has the index of 220.

[8]:
```
print(tweet_df.loc[220].text)
```

```
#TopListAngels  Ready For BO  @malangangels  Malang  dan  by DM
https://t.co/vS1Oukm3yq
```

Let's see whether we can find near duplicates of this Tweet.

[9]:
```
tweet_df['sim'] = tweet_df.text.apply(lambda x:␣
 ↪shingling_jaccard_similarity(tweet_df.loc[220].text, x, 3))
tweet_df.sort_values('sim', ascending=False).head(10)
```

[9]:

	text	sim
220	#TopListAngels Ready For BO @malangangel…	1.000000
2342	RT @dunk_sl: #TopListAngels Ready For BO …	0.750000
4906	RT @dunk_sl: #TopListAngels Ready For BO …	0.521739
2280	RT @dunk_sl: #TopListAngels Ready For BO …	0.521739
2323	#TopListAngels Ready For BO @VaniaBDG_2 …	0.391304
6109	#TopListAngels Ready For BO @Lanci_Mevit…	0.375000
3651	RT @dunk_sl: #TopListAngels Ready For BO …	0.346154
5118	RT @dunk_sl: #TopListAngels Ready For BO …	0.346154
6575	Case of the Monday Blues? Nothing a KNOW Bette…	0.000000
6577	RT @LunnMiss: #TwinPeaks «Fire Walk With M…	0.000000

[10]:
```
near_dups = tweet_df.sort_values('sim', ascending=False).head(10)
for x in list(near_dups.text):
    print(x)
```

```
#TopListAngels  Ready For BO  @malangangels  Malang  dan  by DM
https://t.co/vS1Oukm3yq
RT @dunk_sl: #TopListAngels  Ready For BO  @malangangels  Malang  dan
 by DM https://t.co/dV3ibBQhar
RT @dunk_sl: #TopListAngels  Ready For BO  @malangvhi  Malang  dan
by DM https://t.co/tyqGu1id2S
RT @dunk_sl: #TopListAngels  Ready For BO  @KimiMlg  Malang  dan  by
DM https://t.co/1SIagPPTAP
#TopListAngels  Ready For BO  @VaniaBDG_2  Bandung  dan  by DM
https://t.co/82ZT2nv2Py
#TopListAngels  Ready For BO  @Lanci_Mevita  Expo Cirebon  dan  by DM
https://t.co/KCyJE4cqH3
RT @dunk_sl: #TopListAngels  Ready For BO  @AmorPutri96  Bandung  dan
 by DM https://t.co/GHNSo7Qc1U
RT @dunk_sl: #TopListAngels  Ready For BO  @AmorPutri96  Bandung  dan
 by DM https://t.co/XeNo4Dal2A
Case of the Monday Blues? Nothing a KNOW Better cookie &amp;amp; cup of joe can't
fix.  #CookiesandCoffee #yummy
RT @LunnMiss: #TwinPeaks  «Fire Walk With Me» rt.Dan May @Kyle_MacLachlan
  @mfrost11  @DAVID_LYNCH @LynchFoundation https://…
```

Indeed, we see that the top-8 Tweets look very similar! In fact, many of them are Retweets of the original Tweet, with only small variations on the user handler and/or URLs - so they are indeed near-duplicates. This concludes this assignment.

You have probably noted that although shingling is faster than edit distance, computing the similarity to every other Tweet can still still time consuming (and not necessary if we only care about near-duplicates). In some applications you'll have to compute the similarity between every pair of text, which is even more inefficient. Sometimes we have to compare very long sequences, too (the total length of the human genome is over 3 billion base pairs). There are tricks to greatly speed up similarity computation (by only comparing a fingerprint of the shingles and only evaluating promising pairs), known as MinHash and Locality Sensitive Hashing (LSH), which are usually used in combination with shingling. They are beyond the scope of this course, but we encourage you to read on (http://infolab.stanford.edu/~ullman/mmds/ch3.pdf) and consider using them when you have a large data set in hand! You may find an open-source implementation at https://github.com/ekzhu/datasketch.